kemdiCode MCP: Lorenz Context Compaction, Cluster Bus and Multi-Mind Cognitive Architecture

Dawid Irzyk, MSc Eng – Kemdi Sp. z o.o., Poland – dawid@kemdi.pl

***Abstract*—Large language models used as software engineering assistants must compress multi-step reasoning chains to fit fixed context windows. Naive compression discards structurally critical steps, causing subsequent reasoning to diverge. This paper presents three chaos-theory-based algorithms for information-theoretic context compaction: (i) Poincaré section phase detection using Jensen-Shannon divergence to locate topic transitions with maximum mutual information, (ii) Lorenz attractor orbit compression using TF-IDF cosine similarity matrices to detect and collapse repeating reasoning cycles, and (iii) perturbation impact analysis measuring each item's contribution via leave-one-out JSD. We also introduce *The Nine Minds*, a cognitive architecture of nine specialized reasoning agents (Socratic Interviewer, Ontologist, Seed Architect, Evaluator, Contrarian, Hacker, Simplifier, Researcher, Architect), loaded on-demand and composed dynamically for multi-perspective analysis. These components run within a distributed cognition system: a two-layer event bus with anti-amplification guarantees, eight persistent cognition stores with reactive cross-linking, and a multi-cluster LLM orchestration layer. The system is validated with 618 passing tests across 33 test files.**

***Index Terms*—context compaction, Lorenz attractor, Poincaré section, Jensen-Shannon divergence, chaos theory, cognitive architecture, large language models, multi-agent systems, distributed cognition, dialectical reasoning**

I. Introduction

AI-ASSISTED software engineering tools built on large language models must balance reasoning depth against context capacity. As chains of thought grow through iterative debugging, architectural exploration, or multi-step refactoring, the fixed context window forces lossy compression. Selecting *which* reasoning steps to retain maps directly to sensitivity to initial conditions in chaotic dynamical systems.

In Lorenz's meteorological model [1], infinitesimal perturbations to initial conditions produce exponentially divergent trajectories. Context compaction is the inverse problem: given a trajectory (a chain of thoughts), find the minimal compression that preserves the system's qualitative behavior. Removing a structurally critical thought causes subsequent reasoning to diverge.

We formalize this insight through three algorithms:

(i) *Phase detection via Poincaré sections* [2]: Identify topic transitions in thinking chains using consecutive Jensen-Shannon divergence [6]. Phase boundaries carry maximum mutual information and must survive compaction.

(ii) *Orbit compression via attractor cycle detection*: Detect repeating reasoning patterns (analogous to orbits around the Lorenz attractor's lobes) using TF-IDF cosine similarity matrices, and collapse duplicate cycles while retaining the first occurrence.

(iii) *Perturbation impact via leave-one-out analysis*: Measure each thought's structural contribution to the information landscape by computing JSD between the full context and the context with that item removed. High-impact items are causal anchors.

The paper also presents *The Nine Minds*, a cognitive architecture of nine specialized reasoning agents, and the supporting infrastructure: eight persistent cognition stores with reactive cross-linking, a two-layer event bus with anti-amplification guarantees, and the Magistrale multi-cluster LLM orchestration layer with four aggregation strategies and self-regulating multi-pass execution.

Section II surveys related work. Section III presents the compaction framework. Section IV introduces The Nine Minds. Section V describes the cognition subsystem. Section VI details the event bus. Section VII covers distributed LLM orchestration. Section VIII describes the agentic execution model. Section IX covers security hardening. Section X presents the evaluation. Section XI concludes.

II. Related Work

Context management. Recursive summarization [19] compresses conversation history but discards structural information. Retrieval-augmented generation [20] sidesteps the problem via external vector stores but cannot preserve causal reasoning chains. Attention sinks [21] maintain initial tokens as anchors, observing that removing them degrades perplexity. Our approach treats the thinking chain as a dynamical trajectory and applies chaos theory to identify structurally essential elements.

Multi-agent coordination. AutoGen [24] enables multi-agent conversation through conversable agents. MetaGPT [25] introduces role-based coordination with standardized operating procedures. CrewAI [26] provides task-oriented agent orchestration. Li et al. [27] survey workflow patterns. Guo et al. [28] provide a taxonomy of architectures. Our system operates at the protocol level via the Model Context Protocol [12], [13], exposing coordination primitives as tools rather than implementing a fixed agent framework.

Chaos theory in machine learning. Pascanu et al. [22] demonstrated that vanishing and exploding gradients in recurrent networks are manifestations of Lyapunov exponents in the network's dynamical system. Vogt et al. [23] used Lyapunov exponents to characterize transformer stability. Our contribution applies chaos-theoretic concepts not to model internals but to the *reasoning output*, treating thinking chains as trajectories in a semantic phase space.

Cognitive architectures. Munro's Ouroboros [39] introduced a specification-first system with nine cognitive agent

modes (Socratic Interviewer, Ontologist, Seed Architect, Evaluator, Contrarian, Hacker, Simplifier, Researcher, and Architect) composed within a self-improving loop driven by convergence detection and ambiguity scoring. Kahneman's dual-process theory [31] distinguishes fast heuristic and slow analytical reasoning. De Bono's Six Thinking Hats [32] introduced role-based thinking modes. Mellers et al. [33] showed that adversarial collaboration improves epistemic outcomes. Our Nine Minds build on Munro's taxonomy, adding persistent cognition integration, cluster-level composition via Magistrale, and Lorenz-aware context compaction.

Protocol foundations. The Model Context Protocol [12] generalizes the Language Server Protocol [14] from language-specific operations to arbitrary tool invocations for AI assistants. Our system implements MCP with extensions for distributed cognition.

III. Lorenz-Inspired Context Compaction

A. Theoretical Foundation

The Lorenz system [1]:

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z \tag{1}$$

with canonical parameters $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, exhibits three properties that map to context compaction:

1) **Sensitivity to initial conditions.** The maximal Lyapunov exponent $\lambda_1 \approx 0.906$ [4] implies perturbation growth $\|\delta(t)\| \sim \|\delta_0\| e^{\lambda_1 t}$. Removing a critical thought (perturbing the initial cognitive state) causes exponential divergence in subsequent reasoning.
2) **Strange attractor structure.** Trajectories orbit two lobes, tracing similar but non-identical paths. AI reasoning exhibits analogous behavior: revisiting the same conceptual territory from slightly different angles, producing compressible redundancy.
3) **Lobe transitions.** Rapid transitions between attractor lobes correspond to topic switches in thinking chains. These transitions carry maximum information about the trajectory's qualitative structure.

B. Phase Detection via Poincaré Sections

In dynamical systems, a Poincaré section is a codimension-one submanifold transverse to the flow, reducing continuous dynamics to a discrete return map [2], [3]. We apply this concept to thinking chains: the "flow" is the sequence of thoughts, and "sections" are points where the semantic content changes direction.

Definition 1 (Phase Boundary): Given a thinking chain $\mathcal{T} = (t_1, t_2, \ldots, t_n)$ where each t_i has content c_i, a *phase boundary* exists between t_i and t_{i+1} if:

$$\mathrm{JSD}\big(P(c_i) \| P(c_{i+1})\big) > \tau_{\text{phase}} \tag{2}$$

where $P(c)$ is the token probability distribution of content c and $\tau_{\text{phase}} = 0.35$ is the detection threshold.

The Jensen-Shannon divergence [6] is the symmetrized, bounded variant of the Kullback-Leibler divergence [7]:

$$\mathrm{JSD}(P \| Q) = \frac{1}{2} D_{\mathrm{KL}}(P \| M) + \frac{1}{2} D_{\mathrm{KL}}(Q \| M) \tag{3}$$

where $M = \frac{1}{2}(P + Q)$ and $\mathrm{JSD} \in [0, \ln 2]$. Unlike KL divergence, JSD is always finite and symmetric, making it suitable for comparing empirical token distributions which may have non-overlapping support.

The token probability distribution $P(c)$ is computed by tokenizing content (lowercased, stop words removed, minimum 2 characters), computing term frequencies, and normalizing. This is equivalent to the unigram language model [9].

Property 1 (Phase Boundary Information Maximum): At a phase boundary b_k, the mutual information between the content and the latent "topic" variable is locally maximal:

$$I(c_{b_k}; \text{topic}) \geq I(c_j; \text{topic}) \quad \forall j \in [b_{k-1} + 1, b_{k+1} - 1] \tag{4}$$

This follows from the observation that high JSD between consecutive thoughts indicates maximally different token distributions, meaning the boundary thought carries the most discriminative information about the topic transition. The mutual information is approximated via:

$$I(X; Y) \approx H(X) + H(Y) - H(X, Y) \tag{5}$$

where H is the Shannon entropy [5]: $H(P) = -\sum_x P(x) \log_2 P(x)$.

1) Preservation Strategy: Phase detection identifies thoughts to *preserve* during compaction:

- Trajectory endpoints (first and last thoughts).
- All phase boundary thoughts (maximum information at transitions).
- The highest-confidence thought within each phase segment.

C. Orbit Compression via Attractor Cycle Detection

In the Lorenz system, trajectories orbit around two attractor lobes, each orbit qualitatively similar to the previous. In AI reasoning, an *orbit* is a repeating pattern where the model revisits the same conceptual territory.

Definition 2 (Orbital Cycle): A subsequence $(t_s, t_{s+1}, \ldots, t_{s+L-1})$ of length L is an *orbital cycle* if there exists a subsequent subsequence $(t_{s+L}, \ldots, t_{s+2L-1})$ such that:

$$\frac{1}{L} \sum_{k=0}^{L-1} \mathrm{sim}(t_{s+k}, t_{s+L+k}) \geq \tau_{\text{orbit}} \tag{6}$$

where $\mathrm{sim}(t_i, t_j)$ is the TF-IDF cosine similarity [8] and $\tau_{\text{orbit}} = 0.5$.

1) Detection Algorithm: The TF-IDF cosine similarity [8] between thoughts t_i and t_j is:

$$\mathrm{sim}(t_i, t_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{|\mathbf{v}_i| \cdot |\mathbf{v}_j|} \tag{7}$$

where $\mathbf{v}_i$ is the TF-IDF vector with $\mathrm{TF}(w, t) = \mathrm{count}(w, t)/|t|$ and $\mathrm{IDF}(w) = \log(N/\mathrm{df}(w))$. Constants: $L_{\min} = 2$, $L_{\max} = 10$, $R_{\min} = 2$. The greedy search proceeds from longest to shortest cycles, preventing shorter cycles from fragmenting longer ones.

Algorithm 1: Orbit Compression

Input : thoughts $\mathcal{T} = (t_1, \ldots, t_n)$, threshold τ
Output : keep indices $\mathcal{K}$, prune indices $\mathcal{P}$

$\mathbf{S} \leftarrow$ pairwise TF-IDF cosine similarity matrix;
claimed $\leftarrow \emptyset$;
for $L \leftarrow \min(L_{\max}, \lfloor n/2 \rfloor)$ **to** $L_{\min}$ **do**
  for $s \leftarrow 0$ **to** $n - L \cdot R_{\min}$ **do**
    if $\exists k \in [s, s+L) : k \in$ claimed **then**
      continue
    $(r, \bar{S}) \leftarrow \text{detectRepetitions}(\mathbf{S}, s, L, \tau)$;
    if $r \geq R_{\min}$ **then**
      claimed $\leftarrow$ claimed $\cup \{s, \ldots, s + r \cdot L - 1\}$;
      Keep first cycle $[s, s+L)$, prune copies;
$\mathcal{P} \leftarrow$ all pruned indices;
$\mathcal{K} \leftarrow \{1, \ldots, n\} \setminus \mathcal{P}$;

D. Perturbation Impact Analysis

Drawing on the Lorenz system's sensitivity to initial conditions, we quantify each thought's structural importance via leave-one-out analysis:

Definition 3 (Perturbation Impact): For a context set $\mathcal{C} = \{t_1, \ldots, t_n\}$, the perturbation impact of item t_k is:

$$\Pi(t_k) = \text{JSD}\big(P(\mathcal{C}) \| P(\mathcal{C} \setminus \{t_k\})\big) \tag{8}$$

where $P(\mathcal{C})$ is the aggregate token distribution over all items.

Items with high Π are *causal anchors*: their removal maximally perturbs the information landscape, analogous to critical initial conditions in the Lorenz system. Items with $\Pi \approx 0$ contribute content already represented by other items and can be safely evicted.

Property 2 (Perturbation Bounds): For uniform context (identical items), $\Pi(t_k) = 0$ for all k. For maximally diverse context, Π correlates with information density:

$$\rho(t_k) = \frac{H(P(t_k))}{\log_2 |V(t_k)|} \tag{9}$$

where $|V(t_k)|$ is the vocabulary size of item t_k.

E. Integrated Compaction Pipeline

The three algorithms compose into a pipeline:

1) **Phase detection**: Mark phase boundary thoughts as *protected*.
2) **Orbit compression**: Within each phase, detect and collapse orbital cycles.
3) **Perturbation ranking**: Rank remaining unprotected thoughts by Π. Evict lowest-impact thoughts until the context budget is met.

This composition guarantees that: phase boundaries (maximum-information transitions) are never evicted; redundant orbits are collapsed before perturbation ranking, preventing duplicate content from inflating impact scores; and the final eviction uses information-theoretic ranking rather than heuristic recency or frequency.

F. CTC-Inspired Extensions

The cognition module includes additional algorithms inspired by concepts from general relativity:

Fixed-point compaction. Iteratively compacts a thinking chain until convergence:

$$|C_{i+1}| \geq |C_i| \cdot (1 - \epsilon), \quad \epsilon = 0.05 \tag{10}$$

This mirrors the Novikov self-consistency principle [10]: the compacted result is a fixed point that regenerates itself under further compaction.

Gravitational TTL. Time-to-live based on graph proximity to the active intent:

$$\text{TTL}(t_k) = \text{TTL}_{\text{base}} \cdot \left(1 + \gamma \cdot \frac{1}{1 + d(t_k)}\right) \tag{11}$$

where $d(t_k)$ is the graph distance to the active intent and $\gamma = 1.5$ is the coupling constant. Items close to the intent receive up to $2.5\times$ the base TTL, analogous to gravitational time dilation near massive objects.

Causal DAG analysis. Constructs a directed acyclic graph of dependencies between cognition items and computes the causal core (the minimal set needed to reconstruct all others) via transitive reduction [11].

IV. The Nine Minds: Cognitive Architecture

The compaction pipeline (Section III) addresses *what to remember*; The Nine Minds address *how to think*. Each Mind is a specialized reasoning agent with a distinct epistemological mode, loaded on-demand and composed dynamically as problems evolve.

The Nine Minds architecture is inspired by *Ouroboros* [39], a specification-first AI development system by Harry Munro that introduced nine cognitive agent modes with Socratic methodology and ontological analysis. Our implementation adapts these concepts within a persistent cognition subsystem, adding on-demand loading, cluster-level composition patterns, and integration with the Lorenz compaction pipeline.

A. Theoretical Foundations

The design draws on four intellectual traditions:

1) **Ouroboros specification-first methodology** [39]: Munro's system pioneered nine specialized cognitive agents (from Socratic Interviewer through Ontologist to Seed Architect) within a self-improving loop where evaluation feeds back as input. Our Nine Minds adapt this taxonomy for persistent distributed cognition.
2) **Socratic dialectic** [30]: Knowledge emerges through systematic questioning, not assertion. The Socratic Interviewer Mind embodies this: it produces only questions, never solutions.
3) **Dual-process theory** [31]: Human cognition alternates between fast heuristic (System 1) and slow analytical (System 2) modes. The Nine Minds extend this to nine distinct cognitive modes, selectable based on problem characteristics.
4) **Adversarial collaboration** [33]: Intellectual progress requires structured disagreement. The Contrarian Mind

Table I
THE NINE MINDS: COGNITIVE MODES

Mind	Mode	Core Question
Socratic	Interrogative	"What are you assuming?"
Ontologist	Classificatory	"What IS this, really?"
Seed Architect	Crystallizing	"Is this complete and unambiguous?"
Evaluator	Verificatory	"Did we build the right thing?"
Contrarian	Adversarial	"What if the opposite were true?"
Hacker	Lateral	"What constraints are actually real?"
Simplifier	Reductive	"What's the simplest thing that could work?"
Researcher	Evidential	"What evidence do we actually have?"
Architect	Structural	"If we started over, would we build it this way?"

formalizes this by always arguing the opposite position, steelmanning [34] the counter-argument.

B. Mind Taxonomy

Each Mind operates under strict behavioral constraints encoded in its system prompt. The Socratic Interviewer produces only questions, never code or solutions. The Evaluator follows a mandatory 3-stage protocol (correctness, completeness, fitness). The Researcher classifies every claim as EVIDENCE, OPINION, or HEARSAY. These constraints are invariants enforced by the system prompt architecture, not guidelines.

C. On-Demand Loading

Minds are *lazy-loaded*: no Mind is instantiated until explicitly invoked. This follows the principle of cognitive economy [35]: categories (here, reasoning modes) are activated only when needed, minimizing baseline resource consumption. The loading mechanism mirrors the lazy tool registration pattern used throughout the system (`registerLazyTool()`).

Definition 4 (Mind Invocation): A Mind M_i is invoked by specifying its type as the `agent` parameter in any AI-calling tool (`ask-ai`, `agent-orchestrate`, `multi-prompt`, `consensus-prompt`, `mind-chain`). The Mind's system prompt, temperature, and token budget override the base agent defaults.

D. Composition Patterns

The Nine Minds are designed for sequential composition, invoking multiple Minds on the same problem for multi-perspective analysis:

Dialectical Progression. Socratic → Ontologist → Seed Architect. Question assumptions, identify essence, then crystallize specifications. This mirrors the Hegelian dialectic [36]: thesis (question), antithesis (essence), synthesis (specification).

Adversarial Review. Architect → Contrarian → Evaluator. Propose structure, challenge it adversarially, then verify the survivor. This implements a form of red-team/blue-team methodology applied to design [37].

Evidence-Based Design. Researcher → Hacker → Simplifier. Gather evidence, find unconventional paths, then simplify. This follows the lean startup principle of measure → experiment → build [38].

These compositions are automated by the `mind-chain` tool, which executes a sequence of Minds where each receives the accumulated output from all previous Minds. Predefined chains (*dialectical*, *adversarial*, *evidence*, *full-review*) can be invoked in a single call, with an optional synthesis step that combines all perspectives into a unified recommendation.

Table II
EIGHT COGNITION STORES INTERCONNECTED VIA COGNITIONCROSSLINKER. ALL LINKS ARE BIDIRECTIONAL AND REDIS-BACKED.

Store	Persistence	Cross-Links To
Decision Journal	Redis sorted set	Confidence, Intent, Self-Critique
Confidence Tracker	Redis hash	Decision, Error Patterns
Intent Tracker	Redis hash	Decision, Self-Critique
Error Patterns	Redis sorted set	Confidence, Mental Model
Self-Critique	Redis list	Decision, Intent
Mental Model	Redis hash	Error Patterns, Decision
Handoff Store	Redis hash	Intent, Decision
Context Budget	In-memory + Redis	All stores (read-only)

E. Integration with Cognition Subsystem

Each Mind invocation generates cognition artifacts that persist across sessions:

- Socratic questions feed the *Decision Journal* as decision preconditions.
- Ontologist classifications create nodes in the *Knowledge Graph*.
- Evaluator verdicts update *Confidence Tracker* scores.
- Contrarian counter-arguments are recorded as *Decision Journal* alternatives.
- Researcher findings populate *Error Pattern* and *Mental Model* stores.

Mind outputs are not ephemeral; they enrich the persistent cognition layer (Section V) and become inputs for future reasoning across sessions.

V. PERSISTENT COGNITION SUBSYSTEM

The cognition layer provides persistent AI self-awareness through eight stores interconnected by a cross-linking mechanism. All stores use Redis [15] for persistence with configurable TTL.

A. Store Descriptions

Decision Journal. Records structured decisions with options, reasoning, chosen path, and deferred outcome tracking. Supports retrospective analysis of decision quality.

Confidence Tracker. Maintains calibrated confidence scores per domain with Expected Calibration Error (ECE) [18] monitoring. Tracks trends to detect systematic over- or under-confidence.

Intent Tracker. Hierarchical goal management with drift detection using TF-IDF bigram similarity. When current work diverges from stated goals, triggers automatic self-critique.

Error Pattern Database. Cross-session error database with symptom matching, root cause tracking, and fix suggestions. Uses sorted sets for temporal ordering and merge-on-collision for duplicate patterns.

Self-Critique Store. Post-task reflection with structured lessons. Lessons are cross-linked to error patterns for reinforcement.

Mental Model Store. Component-relationship architecture maps with invariant checking, dependency chain analysis, and impact analysis for proposed changes.
Handoff Store. Auto-enriched session snapshots capturing active tasks, pending decisions, mental models, and error patterns for session recovery after context compaction.
Context Budget Manager. Token estimation with perturbation-based relevance scoring. Items with high Π (from Section III-D) are flagged as eviction-resistant anchors.

B. Reactive Cross-Linking

The CognitionCrossLinker creates bidirectional Redis links between stores. Nine reactive event handlers create autonomous feedback loops:

- `decision:recorded` → auto-creates confidence record
- `confidence:low` → triggers intent drift detection
- `error:recorded` → scans recent decisions for root cause
- `critique:lesson` → cross-links to matching error patterns
- `intent:drifted` → triggers self-critique
- `handoff:created` → links active mental models
- `model:stale` → flags dependent decisions and intents

C. Agent Ranking

Agent performance is tracked using a composite score with five tiers (bronze through diamond):

$$\text{Score} = 0.4 \cdot S_r + 0.3 \cdot C_t + 0.2 \cdot A_s + 0.1 \cdot C_o \tag{12}$$

with exponential temporal decay $\text{Score}(t) = \text{Score}_0 \cdot 0.95^{\Delta t/\tau}$ preventing stale scores from persisting indefinitely.

VI. Two-Layer Event Bus Architecture

The communication infrastructure has two layers connected by anti-amplification bridges.

A. L_3: ClusterBus

Inter-cluster communication via Redis Pub/Sub [15] with 18 signal types, four send modes (unicast, broadcast, routed via MetaRouter, multicast), and three subsystems:

1) SignalFlowController: Governs throughput through per-signal-type rate limiting, a circuit breaker [17] implementing the closed → open → half-open automaton, priority-based filtering, and backpressure with configurable buffering.

2) Dual-Layer Deduplication: A bounded set of 2,000 recent signal IDs provides exact deduplication. Overflow is handled by a Bloom filter [16] configured for 20,000 items at 0.1% false-positive rate:

$$m = -\frac{n \ln p}{(\ln 2)^2}, \quad k = \frac{m}{n} \ln 2 \tag{13}$$

3) MetaRouter: Tag-based signal routing supporting four match modes (exact, wildcard, prefix, regex) with AND/OR boolean logic, priority ordering, and terminal rules. Regex patterns are capped at 200 characters for ReDoS prevention.

B. L_1: GlobalEventBus

In-process event emitter with namespaced events (`module:category:action`), async dispatch via `queueMicrotask`, chain depth limit of 8, and Redis bridge for cross-session propagation with retry (max 2 retries, 200ms exponential backoff).

C. Anti-Amplification Guarantees

Invariant 1 (Signal Termination): For any signal s, the total copies across all layers is bounded by:

$$|C(s)| \leq H_{\max} \cdot D_{\max} = 5 \times 8 = 40 \tag{14}$$

where $H_{\max} = 5$ is the maximum bridge hops and $D_{\max} = 8$ is the maximum event chain depth. In practice, source prefix guards and deduplication reduce this to $|C(s)| \leq 3$.

Four mechanisms enforce termination: hop counter (max 5) on bridged signals, source prefix filtering to prevent echo, per-bridge deduplication set, and chain depth limiting in the GlobalEventBus.

VII. LLM Magistrale

The Magistrale dispatches prompts across multiple cluster nodes with four aggregation strategies and self-regulating multi-pass execution.

A. Aggregation Strategies

First-wins. Returns the first successful response, canceling others. Optimized for latency.
Best-of-n. Dispatches to n clusters, scores responses using:

$$S = 0.45 \cdot Q + 0.25 \cdot D + 0.15 \cdot T - 0.15 \cdot L \tag{15}$$

where Q is quality (from pass report), D is content depth ($\log_{10}$-scaled length), T is token efficiency (completion/prompt ratio), and L is normalized latency penalty.
Consensus. Requires agreement between responses using weighted TF-IDF cosine similarity (Section III-C) with threshold 0.3. Each response's vote weight is proportional to its quality score squared, amplifying high-quality contributions. Applicable to architecture decisions where correctness outweighs speed.
Fallback-chain. Sequential execution with priority ordering. First successful response terminates the chain. Optimized for availability.

B. Lorenz Prompt Compression

Before dispatching to remote clusters, the Magistrale applies orbit compression (Section III-C) to the prompt. Prompts exceeding 3,000 characters are split into paragraph-sized blocks, analyzed for repeating patterns via the TF-IDF cosine similarity matrix, and redundant cycles are removed. This reduces token cost on resource-constrained cluster nodes without losing semantic content.

C. PassController

Three strategies govern multi-pass execution:

- *min-passes*: The LLM self-assesses task complexity and declares minimum passes, capped to a budget parameter.
- *quality-target*: Iterates until quality $Q_i \geq Q_{\text{target}}$ or budget exhausted.
- *fixed*: Executes exactly N passes without assessment.

D. Cluster-Internal Orchestration

When a cluster receives an `llm:request` signal with an `orchestrate` payload, it spawns a full agentic loop (Section VIII) *inside* the cluster node rather than forwarding a simple LLM call. The orchestration supervisor has access to all kemdiCode tools and iterates autonomously:

1) The Magistrale dispatches an `llm:request` with `orchestrate.{agent, maxIterations, allowedTools, enableCognition}`.
2) The receiving cluster's `llm:request` handler detects the `orchestrate` field and calls `executeAgenticLoop()` with the specified agent persona (`plan`, `build`, `explore`, or `general`).
3) The agent reasons over the prompt, calls tools (e.g. `find-definition`, `semantic-search`, `thinking-chain`), and iterates up to `maxIterations` cycles.
4) Upon completion, the cluster sends an `llm:response` signal containing the agent's final answer, iteration count, and total tool calls.

This enables distributed autonomous analysis: a hub cluster can dispatch "find race conditions in module X" to three worker clusters, each of which independently explores the codebase, calls code intelligence tools, and returns a reasoned answer — all without the hub needing to coordinate individual tool calls.

Timeout is calculated as $\min(\texttt{maxIterations}, 30) \times 15\,\text{s}$ to prevent unbounded execution. Tool access can be restricted via `allowedTools`/`blockedTools` whitelists, and cognition tools (decision journal, confidence tracker, etc.) are optionally enabled via `enableCognition`.

VIII. Agentic Execution Model

The agentic loop implements autonomous tool execution with full orchestration traceability. Resource constraints:

- **Tool access**: All kemdiCode tools are available to agents by default (read-only operations, kanban, thinking chains). Access can be customized per-agent via `allowedTools`/`blockedTools` parameters.
- **Sub-agent spawning**: Maximum depth 2 (configurable), global budget of 10 concurrent sub-agents. Slot acquisition is synchronous (no TOCTOU races in single-threaded execution).
- **Time budget**: Hard deadline of 10 minutes per orchestration (configurable via `MCP_ORCH_TIMEOUT` or `timeoutMs` parameter). Checked at each iteration boundary before starting new AI calls.
- **Parallel execution**: 2–10 agents execute concurrently via `Promise.allSettled`. Each receives a unique `orchestrationId`; sub-agents reference their parent via `parentOrchestrationId`.
- **Lorenz-aware context management**: In function-calling mode, conversation history is pruned using perturbation impact ranking (Section III-D) instead of FIFO eviction — high-information messages are preserved even if old. In text-based mode, accumulated tool results are periodically compressed via orbit detection (Section III-C), removing repeating result patterns mid-loop.
- **Live monitoring**: Orchestration status (iteration, tool calls, last result) is tracked in an in-memory cache with asynchronous Redis persistence, queryable via `GET /orchestrations` HTTP endpoint.
- **Cognition integration**: All cognition records (decisions, confidence, intents, errors, critiques, handoffs) carry `orchestrationId` for end-to-end traceability across nested agent hierarchies.
- **File context**: `@path` syntax injects file contents into prompts.
- **Session isolation**: Per-session state via `AsyncLocalStorage` [29], propagating through async chains to sub-agents.

IX. Security Hardening

- HMAC-SHA256 authentication on ClusterBus signals with constant-time comparison.
- Bloom filter deduplication (20K items, 0.1% FPR) prevents signal replay.
- Prototype pollution protection in all Redis JSON deserialization paths.
- Zod schema validation on all tool inputs, preventing injection.
- Path traversal protection via `validatePathSafe()`.
- ReDoS prevention: regex pattern length capped at 200 characters.
- Client timeout guards: 30s on `client-sampling` and `client-elicit`.
- Thinking chain depth limit: 500 steps maximum.

X. Evaluation

The system is validated through 618 tests across 33 test files: unit tests (cognition stores, Lorenz compaction algorithms, bus components, security hardening), integration tests (multi-agent communication, session sharing, tool chains), performance benchmarks (signal throughput, concurrent operations, scalability), and resilience tests (Redis degradation, edge cases, error handling).

Key results:

- SignalFlowController: >50,000 signals/sec without policies, >20,000 with rate limiting.
- Circuit breaker: >100,000 ops/sec.
- Bloom filter: zero false negatives across all test scenarios.
- Lorenz compaction: phase detection correctly identifies topic transitions in multi-phase debugging chains; orbit compression detects and prunes repeated analysis patterns; CTC fixed-point converges within 3 iterations on realistic data.

Client compatibility is verified with Claude Code (primary), Cursor, KiroCode, and RooCode.

XI. Conclusion

Chaos theory and dynamical systems provide a rigorous framework for AI context compaction. Poincaré section phase detection identifies information-dense topic transitions; Lorenz attractor orbit compression eliminates redundant reasoning cycles; perturbation impact analysis quantifies each thought's structural importance. These algorithms replace heuristic recency-based eviction with information-theoretic compression grounded in established mathematics.

The Nine Minds cognitive architecture provides specialized reasoning modes (Socratic questioning, adversarial challenge, structural analysis, among others) that compose dynamically for multi-perspective engineering analysis. Each Mind integrates with the persistent cognition subsystem, so insights accumulate across sessions.

The distributed cognition infrastructure (eight reactive stores with cross-linking, a two-layer event bus with formal termination guarantees, and multi-cluster LLM orchestration) supports persistent AI self-awareness across session boundaries.

Future work: embedding-based similarity replacing TF-IDF for higher-fidelity orbit detection, automatic Mind selection based on problem characteristics, cross-project knowledge graph learning, and adaptive compaction thresholds tuned by downstream task performance.

References

[1] E. N. Lorenz, "Deterministic nonperiodic flow," *J. Atmospheric Sci.*, vol. 20, no. 2, pp. 130–141, Mar. 1963. doi:10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2

[2] H. Poincaré, "Sur le problème des trois corps et les équations de la dynamique," *Acta Math.*, vol. 13, pp. 1–270, 1890.

[3] S. H. Strogatz, *Nonlinear Dynamics and Chaos*, 2nd ed. Boulder, CO: Westview Press, 2015.

[4] C. Sparrow, *The Lorenz Equations: Bifurcations, Chaos, and Strange Attractors*. New York: Springer-Verlag, 1982. doi:10.1007/978-1-4612-5767-7

[5] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948. doi:10.1002/j.1538-7305.1948.tb01338.x

[6] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inform. Theory*, vol. 37, no. 1, pp. 145–151, Jan. 1991. doi:10.1109/18.61115

[7] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951. doi:10.1214/aoms/1177729694

[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inform. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988. doi:10.1016/0306-4573(88)90021-0

[9] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge: Cambridge Univ. Press, 2008.

[10] I. D. Novikov, "An analysis of the operation of a time machine," *Sov. Phys. JETP*, vol. 68, no. 3, pp. 439–443, 1989.

[11] A. V. Aho, M. R. Garey, and J. D. Ullman, "The transitive reduction of a directed graph," *SIAM J. Comput.*, vol. 1, no. 2, pp. 131–137, 1972. doi:10.1137/0201008

[12] Anthropic, "Introducing the Model Context Protocol," Anthropic Blog, Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[13] Model Context Protocol Authors, "Model Context Protocol Specification, version 2025-11-25," 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[14] Microsoft, Red Hat, and Codenvy, "Language Server Protocol Specification," 2016. [Online]. Available: https://microsoft.github.io/language-server-protocol/

[15] S. Sanfilippo, "Redis: An in-memory data structure store," 2009–2026. [Online]. Available: https://redis.io/

[16] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970. doi:10.1145/362686.362692

[17] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*. Raleigh, NC: Pragmatic Bookshelf, 2007.

[18] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017.

[19] J. Wu *et al.*, "Recursively summarizing books with human feedback," *arXiv:2109.10862*, 2021.

[20] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Inform. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.

[21] G. Xiao *et al.*, "Efficient streaming language models with attention sinks," in *Proc. ICLR*, 2024.

[22] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. ICML*, 2013.

[23] R. Vogt, M. Puelma Touzel, E. Shlizerman, and G. Lajoie, "On Lyapunov exponents for RNNs: Understanding information propagation using dynamical systems tools," *Front. Appl. Math. Statist.*, vol. 8, Art. no. 818799, 2022. doi:10.3389/fams.2022.818799

[24] Q. Wu, G. Bansal *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv:2308.08155*, 2023.

[25] S. Hong, X. Zhuge *et al.*, "MetaGPT: Meta programming for multi-agent collaborative framework," *arXiv:2308.00352*, 2023.

[26] J. Moura, "CrewAI: Framework for orchestrating role-playing autonomous AI agents," 2024. [Online]. Available: https://github.com/crewAIInc/crewAI

[27] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 1, Art. no. 9, Oct. 2024. doi:10.1007/s44336-024-00009-2

[28] T. Guo, X. Chen *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. 33rd IJCAI*, 2024.

[29] Node.js Contributors, "AsyncLocalStorage," Node.js Documentation, 2020–2026. [Online]. Available: https://nodejs.org/api/async_context.html

[30] Plato, *Meno*, ca. 385 BCE. Trans. G. M. A. Grube, Indianapolis: Hackett, 1976.

[31] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.

[32] E. de Bono, *Six Thinking Hats*. Boston: Little, Brown and Company, 1985.

[33] B. Mellers, R. Hertwig, and D. Kahneman, "Do frequency representations eliminate conjunction effects? An exercise in adversarial collaboration," *Psychol. Sci.*, vol. 12, no. 4, pp. 269–275, 2001. doi:10.1111/1467-9280.00350

[34] S. F. Aikin, "Straw man, weak man, hollow man," in *Bad Arguments*, R. Arp *et al.*, Eds. Chichester: Wiley, 2017, pp. 44–48.

[35] E. Rosch, "Principles of categorization," in *Cognition and Categorization*, E. Rosch and B. B. Lloyd, Eds. Hillsdale, NJ: Erlbaum, 1978, pp. 27–48.

[36] G. W. F. Hegel, *Phänomenologie des Geistes*. Bamberg: Goebhardt, 1807. Trans. A. V. Miller, *Phenomenology of Spirit*, Oxford Univ. Press, 1977.

[37] M. Zenko, *Red Team: How to Succeed by Thinking Like the Enemy*. New York: Basic Books, 2015.

[38] E. Ries, *The Lean Startup*. New York: Crown Business, 2011.

[39] H. Munro, "Ouroboros: A Specification-First AI Development System," 2026. [Online]. Available: https://github.com/Q00/ouroboros